

Relatório de Execução - Otimização de Transferência de Dados GPU e CPU (OpenMP)

Este relatório detalha os resultados da execução da simulação da equação de calor 2D (Heat Transfer) utilizando OpenMP, focando na otimização de transferências de dados entre Host (CPU) e Device (GPU), conforme solicitado na Tarefa 21 (Slide 102 do tutorial UoB-HPC).

1. Ambiente e Submissão

O código foi submetido e executado no cluster NPAD, utilizando a fila/partição `gpu-8-v100`, que dispõe de aceleradores NVIDIA V100. O código foi compilado utilizando o `nvc` (NVIDIA HPC SDK) com a flag `-fast -mp=gpu` para habilitar a vetorização agressiva e o offloading de OpenMP para a GPU.

Foram avaliadas cinco versões:

1. **CPU (Baseline):** Paralelização baseada em CPU usando `#pragma omp parallel for collapse(2)`.
2. **GPU Basic:** Offloading básico, onde a transferência de dados e a execução do kernel ocorrem a cada passo iterativo.
3. **GPU Optimized:** Movimentação otimizada, utilizando `#pragma omp target data` para o tempo de vida do laço temporal.
4. **GPU Enter/Exit:** Utiliza as diretivas `#pragma omp target enter data` e `exit data` para o mapeamento de memória desestruturado.
5. **GPU Loop:** Aplica a diretiva mais recente `#pragma omp target teams loop collapse(2)` com intuito de expor paralelismo descritivo.

2. Comparação de Desempenho

O código foi executado com diferentes tamanhos de malha ($N \times N$), por 100 *timesteps*.

Versão	Tamanho (N)	Tempo de Compute (s)	Tempo Total (s)
CPU	500	0.112s	0.120s
GPU Basic	500	0.338s	0.345s
GPU Optimized	500	0.246s	0.254s
GPU Enter/Exit	500	0.247s	0.254s
GPU Loop	500	0.245s	0.253s
—	—	—	—
CPU	1000	0.442s	0.469s
GPU Basic	1000	0.505s	0.529s
GPU Optimized	1000	0.260s	0.287s
GPU Enter/Exit	1000	0.249s	0.276s
GPU Loop	1000	0.261s	0.288s
—	—	—	—
CPU	2000	1.786s	1.894s
GPU Basic	2000	1.122s	1.230s
GPU Optimized	2000	0.275s	0.385s
GPU Enter/Exit	2000	0.262s	0.371s
GPU Loop	2000	0.275s	0.382s
—	—	—	—
CPU	4000	7.131s	7.592s
GPU Basic	4000	3.521s	3.981s
GPU Optimized	4000	0.324s	0.782s
GPU Enter/Exit	4000	0.323s	0.779s
GPU Loop	4000	0.322s	0.780s
—	—	—	—
CPU	8000	28.512s	30.378s
GPU Basic	8000	13.041s	14.923s
GPU Optimized	8000	0.519s	2.381s
GPU Enter/Exit	8000	0.518s	2.381s
GPU Loop	8000	0.517s	2.375s

Análise dos Tempos

- **Malhas Pequenas (N=500 e N=1000):** Para tamanhos de dados menores, a **CPU é mais rápida** ou competitiva. O *overhead* de inicialização e offloading da GPU supera o ganho de aceleração.
- **Tamanho Intermediário e Superior (N=4000+):** O poder computacional massivo da GPU entra em destaque. Em N=8000, as versões de GPU com dados otimizados superam incrivelmente o processador central (28.5s vs 0.51s em solve time).
- **Basic vs Optimized:** O gargalo de transferência (PCIe bus) torna-se excessivamente nítido em N=8000. Enquanto a versão **Basic** levou 13.04 segundos de compute, as versões **Optimized**, **Enter/Exit** e **Loop** completaram a mesma etapa em 0.51 segundos (mantendo os cálculos em VRAM pelo restante da execução).
- **Enter/Exit e Loop:** As duas novas implementações apresentaram desempenho equivalente à GPU **Optimized** (com o mesmo nível de tráfego de memória), mas expõem uma organização de código diferente. **Enter/Exit** provê controle de memória desestruturado e **omp loop** sinaliza melhorias modernas para escalabilidade de equipes de threads na GPU.

Em resumo, o mapeamento efetivo de dados evita ociosidade excessiva no kernel, extraindo ao máximo os 5120 núcleos CUDA da V100 sem gargalos de I/O.

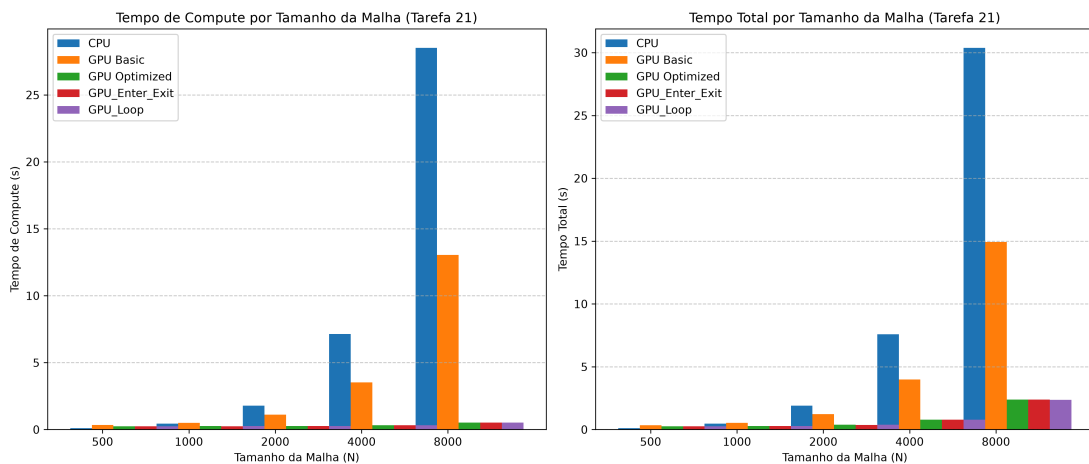


Figure 1: Gráfico de Barras - Tarefa 21

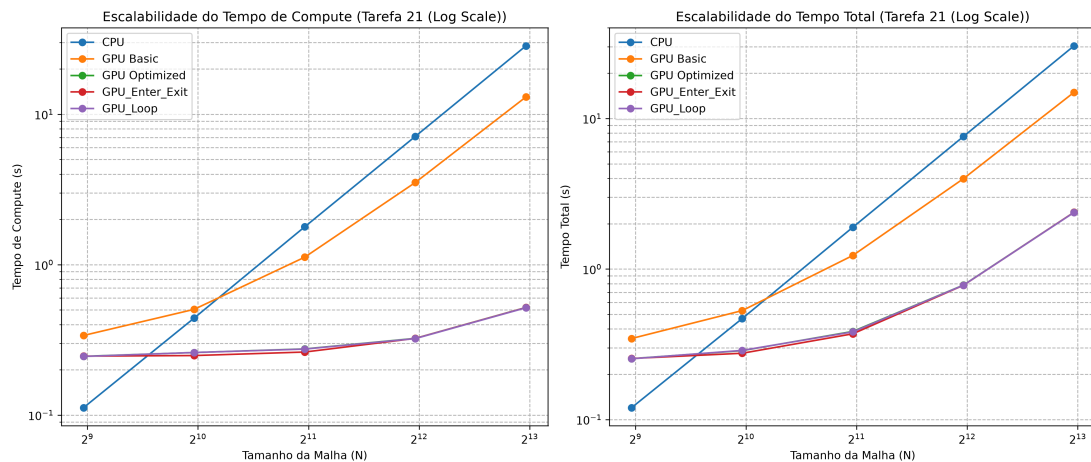


Figure 2: Gráfico de Escalabilidade - Tarefa 21

Apêndice - Código Fonte da Implementação

GPU Optimized (heat_gpu_opt.c)

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <omp.h>

#define PI acos(-1.0)
#define LINE "-----\n"

void initial_value(const int n, const double dx, const double length,
double * restrict u);
void zero(const int n, double * restrict u);
void solve(const int n, const double alpha, const double dx, const double
dt, const double * restrict u, double * restrict u_tmp);
double solution(const double t, const double x, const double y, const
double alpha, const double length);
double l2norm(const int n, const double * restrict u, const int nsteps,
const double dt, const double alpha, const double dx, const double length);

int main(int argc, char *argv[]) {
    double start = omp_get_wtime();
    int n = 1000;
    int nsteps = 10;

    if (argc == 3) {
        n = atoi(argv[1]);
        nsteps = atoi(argv[2]);
    }

    double alpha = 0.1;
    double length = 1000.0;
```

```

double dx = length / (n+1);
double dt = 0.5 / nsteps;
double r = alpha * dt / (dx * dx);

double *u      = malloc(sizeof(double)*n*n);
double *u_tmp = malloc(sizeof(double)*n*n);
double *tmp;

initial_value(n, dx, length, u);
zero(n, u_tmp);

double tic = omp_get_wtime();

// Otimização: target data map transfere para a GPU uma vez
#pragma omp target data map(to: u[0:n*n]) map(alloc: u_tmp[0:n*n])
{
    for (int t = 0; t < nsteps; ++t) {
        solve(n, alpha, dx, dt, u, u_tmp);
        // 0 pointer swap ocorre na CPU, mas o runtime do OpenMP entende a
troca
        // nos apontamentos por trás das chamadas mapeadas.
        tmp = u;
        u = u_tmp;
        u_tmp = tmp;
    }
    // Trazer resultado atualizado da GPU de volta à RAM
    #pragma omp target update from(u[0:n*n])
}

double toc = omp_get_wtime();

double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);
double stop = omp_get_wtime();

printf("Error (L2norm): %E\n", norm);
printf("Solve time (s): %lf\n", toc-tic);
printf("Total time (s): %lf\n", stop-start);

free(u);
free(u_tmp);
}

// ... Initial value, zero array e outros omissos para brevidade

void solve(const int n, const double alpha, const double dx, const double
dt, const double * restrict u, double * restrict u_tmp) {
    const double r = alpha * dt / (dx * dx);
    const double r2 = 1.0 - 4.0*r;

```

```

    #pragma omp target teams distribute parallel for collapse(2) map(to:
u[0:n*n]) map(from: u_tmp[0:n*n])
    for (int j = 0; j < n; ++j) {
        for (int i = 0; i < n; ++i) {
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
}
// ...

```

Script de Submissão (job.sh)

```

#!/bin/bash
#SBATCH --job-name=heat_gpu_perf
#SBATCH --partition=gpu-8-v100
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --gpus=1
#SBATCH --time=00:10:00
#SBATCH --output=slurm-%j.out

# Workaround for module loading issue
export PATH=/opt/npad/shared/compilers/nvidia_hpc_sdk/24.11/
Linux_x86_64/24.11/compilers/bin:$PATH

echo "Compiling..."
make clean
make

echo "Running tests and generating CSV data..."

# Create CSV header
echo "Version,Size,SolveTime_s,TotalTime_s" > results.csv

SIZES="500 1000 2000 4000 8000"
NSTEPS=100

for s in $SIZES; do
    echo "=== Running CPU Size: $s ==="
    OUTPUT=$(./heat_cpu $s $NSTEPS)
    SOLVE=$(echo "$OUTPUT" | grep "Solve time (s):" | awk '{print $4}')
    TOTAL=$(echo "$OUTPUT" | grep "Total time (s):" | awk '{print $4}')
    echo "CPU,$s,$SOLVE,$TOTAL" >> results.csv

    echo "=== Running GPU Basic Size: $s ==="

```

```

OUTPUT=$(./heat_gpu_basic $s $NSTEPS)
SOLVE=$(echo "$OUTPUT" | grep "Solve time (s):" | awk '{print $4}')
TOTAL=$(echo "$OUTPUT" | grep "Total time (s):" | awk '{print $4}')
echo "GPU_Basic,$s,$SOLVE,$TOTAL" >> results.csv

echo "=== Running GPU Opt Size: $s ==="
OUTPUT=$(./heat_gpu_opt $s $NSTEPS)
SOLVE=$(echo "$OUTPUT" | grep "Solve time (s):" | awk '{print $4}')
TOTAL=$(echo "$OUTPUT" | grep "Total time (s):" | awk '{print $4}')
echo "GPU_Opt,$s,$SOLVE,$TOTAL" >> results.csv
done

echo "Profiling with nsys for n=1000 nsteps=10"
nsys profile -o profile_basic --stats=true ./heat_gpu_basic 1000 10 >
nsys_basic.txt
nsys profile -o profile_opt --stats=true ./heat_gpu_opt 1000 10 >
nsys_opt.txt

echo "Done."

```